Block-Filtering Dürr–Hoyer (BFDH): Correlation Between *Filtering Fraction* and *Drop Point* Under a Fixed *Step Budget*

M. Satria Obama

June 20, 2026

Abstract

This paper introduces Block-Filtering Dürr–Hoyer (BFDH), an analytical tool for studying the correlation between the number of elements an oracle flags in the Dürr–Hoyer minimum-finding algorithm (M) and the resulting drop in the threshold value, which I refer to as the *drop point* (DP). BFDH is not meant to replace or compete with the standard Dürr–Hoyer (DH) algorithm or Grover's algorithm; its sole purpose is to characterize the relationship between M and DP. All analysis in this paper is carried out under a step budget (*step budget*) S that is fixed in advance and held constant throughout a given line of analysis: the question being answered is "given a fixed budget of S steps, what filtering fraction α maximizes the ratio of gain to Grover-iteration cost?" In Section 3, I derive the efficiency function for $S = 1$ and show a single balance point at $\alpha^* = 2/3$, giving an 8.87% speedup over standard DH. In Section 5 and beyond, this analysis is extended to S consecutive steps with α held constant, and a closed-form efficiency ratio $R_S(\alpha)$ is derived. The central result of this extension is that the optimum α^* is not a single constant but a function of the step budget S that increases monotonically from $2/3$ (at $S = 1$) toward the structural bound $\alpha = 2$ as $S \to \infty$. This result is established both analytically and through direct numerical experiments.

1 Introduction

Classically, finding the minimum element in an unstructured dataset takes $O(N)$ time in the worst case, since every element has to be checked one by one with no reliable shortcut available without prior knowledge of the data's structure.

Grover's algorithm [1] changed this fundamentally. By exploiting quantum superposition and destructive interference to suppress the amplitudes of incorrect answers, Grover showed that searching for a specific element can be done in $O(\sqrt{N})$ oracle iterations, a quadratic speedup proven optimal for unstructured search.

Dürr and Høyer [2] later adapted this idea to find a minimum element. Their algorithm works as follows: a random value is drawn from the dataset as the initial threshold T, and the Grover oracle is then run to search for elements smaller than T. Each time a smaller element is found, the threshold is updated to that value. This process repeats until no smaller element can be found, yielding the global minimum in $O(\sqrt{N})$ total oracle iterations.

One aspect of the standard DH algorithm that has not been widely explored is how much of the search space the oracle flags at each step. In standard DH, the oracle flags *every* element smaller than T. If there are M such elements, Grover needs $\Theta(\sqrt{N/M})$ iterations at that step. When M is large, the iteration is fast, but the measured value tends to land near the midpoint of the flagged region, so the new threshold ends up close to the previous one. Conversely, if the oracle flags only a small slice of that range, the threshold can drop far more sharply, though at the cost of more Grover iterations for that step.

This paper formalizes that trade-off. We define Block-Filtering Dürr–Hoyer (BFDH) as a variant in which the oracle flags only elements in the range $[0, \alpha T]$, governed by a *filtering fraction* parameter α. BFDH is not proposed as an algorithm that practically outperforms Grover or standard DH; it is an analytical device for studying the relationship between M (number of flagged elements) and DP (*drop point*, i.e., how far the threshold falls).

Framework used in this paper. All analysis here fixes a step budget S in advance and holds it constant throughout a given line of analysis. The question being answered consistently is: *given a fixed budget of S steps, which α maximizes the ratio of gain (DP) to cost (Grover iterations) within that budget?* This differs from the question of "what is the total cost to actually find the minimum with no step limit," which is the classical question in the search-algorithm literature. The two questions sit in different contexts by definition — much as "optimal under a fixed budget of S steps" and "optimal for completing the search outright" are not automatically the same thing. We touch on this distinction briefly in Section 8 where relevant, without making it the main focus, since the distinction itself is fairly easy to grasp; every formula, theorem, and experiment in this paper is defined and should be read within that fixed step-budget framework.

Section 3 derives the analysis for $S = 1$ and finds the balance point $\alpha^* = 2/3$. Section 5 extends this result to general S, and Section 7 proves that the optimum shifts monotonically toward the bound $\alpha = 2$ as S grows. Section 8 presents numerical experiments that verify all of these claims directly.

2 Basic Definitions and Notation

Definition 1 (Problem Parameters)**.** *Let the dataset have N elements with values drawn from a continuous uniform distribution over some interval. At a given step, the running threshold T is the smallest value found so far. The parameter M denotes the number of elements flagged by the oracle (elements with values below the filtering cutoff), and $\alpha = M/T$ is the filtering fraction that determines how aggressively the search space gets blocked off.*

Definition 2 (*Drop Point*, DP)**.** *Drop point (DP) measures how far the threshold falls in one successful BFDH step. If the oracle flags the range* $[0, \alpha T]$*, the value measured after the quantum measurement lands somewhere in* $[0, \alpha T]$*, with an expected value at the midpoint of that range,* $\alpha T/2$*. The expected threshold drop is therefore:*

$$\mathrm{DP} = T - \frac{\alpha T}{2} = T\left(1 - \frac{\alpha}{2}\right). \tag{1}$$

DP is inversely related to α*: the smaller* α *is (the more aggressive the filtering), the larger DP becomes (the further the threshold falls).*

Definition 3 (Grover Iteration Cost)**.** *Following the result of Boyer et al. [3], the number of Grover iterations needed to amplify the amplitude over M flagged elements out of a space of size N is:*

$$I \approx \frac{\pi}{4}\sqrt{\frac{N}{M}} = \frac{\pi}{4}\sqrt{\frac{N}{\alpha T}}. \tag{2}$$

Since N and T stay fixed within a single step, this cost scales inversely with $\sqrt{\alpha}$*: the smaller* α *is, the more iterations are needed. This is the trade-off between DP and Grover cost that sits at the core of this entire analysis.*

3 Analysis at Step Budget $S = 1$

This section derives the efficiency function for *step budget* $S = 1$, which serves as the foundation for the generalizations in later sections. The goal is to find the value of α that yields the best ratio between gain (DP) and cost (Grover iterations).

3.1 Deriving the Function $f(M)$

Efficiency at $S = 1$ is the ratio between what is gained (DP) and what is paid (Grover cost):

$$\mathrm{Eff}(M) = \frac{\mathrm{DP}}{I} = \frac{T - M/2}{\frac{\pi}{4}\sqrt{N/M}}. \tag{3}$$

After simplifying by factoring out the constant $4/\pi \cdot \sqrt{N}$, which does not depend on M:

$$\mathrm{Eff}(M) = \frac{4}{\pi}\sqrt{N} \cdot \left(T - \frac{M}{2}\right)\sqrt{M}. \tag{4}$$

Since what matters here is the location of the M that maximizes $\mathrm{Eff}(M)$, not its absolute value, maximizing equation (4) is equivalent to maximizing:

$$f(M) = \left(T - \frac{M}{2}\right)\sqrt{M}. \tag{5}$$

The function $f(M)$ is a product of two components pulling in opposite directions. The first, $(T - M/2)$, directly represents DP: the smaller M is, the larger this term gets, meaning the threshold falls further — this component wants M as small as possible. The second, $\sqrt{M}$, represents the relative speed of the Grover circuit (the inverse of cost): the larger M is, the larger this term gets, so this component wants M as large as possible. The optimum M^* is the balance point between these two competing pulls.

3.2 Optimum Point M^*

Theorem 1 (Optimum at $S = 1$)**.** *The function* $f(M) = (T - M/2)\sqrt{M}$ *on the domain* $M \in (0, T]$ *attains a global maximum at* $M^* = 2T/3$*.*

Proof. Rewrite $f(M)$ in a form that is easier to differentiate with respect to M:

$$f(M) = T \cdot M^{1/2} - \frac{1}{2}M^{3/2}. \tag{6}$$

The first derivative with respect to M is:

$$f'(M) = \frac{T}{2}M^{-1/2} - \frac{3}{4}M^{1/2} = \frac{T}{2\sqrt{M}} - \frac{3\sqrt{M}}{4}. \tag{7}$$

Setting $f'(M) = 0$ and multiplying both sides by $4\sqrt{M}$:

$$2T - 3M = 0 \implies M^* = \frac{2T}{3}. \tag{8}$$

To confirm that this point is indeed a maximum, the second derivative is computed:

$$f''(M) = -\frac{T}{4}M^{-3/2} - \frac{3}{4}M^{-1/2}. \tag{9}$$

Both terms in equation (9) are negative for all $M > 0$, so $f''(M) < 0$ across the entire domain. The curve $f(M)$ is concave, and the point $M^* = 2T/3$ is a global maximum. □

Numerical verification. Table 1 shows $f(M)$ for a few concrete choices of M with $T = 100$ and $N = 10^6$. $M = 67$ (the integer rounding of $2T/3 \approx 66.67$) yields the highest $f(M)$, consistent with the analytical result.

Table 1: Comparison of $f(M)$ for several values of M ($T = 100$, $N = 10^6$). Standard DH sits at $M = 100$ with $f = 500.0$.

M	$T - M/2$	$\sqrt{M}$	$f(M)$	Status
25	87.5	5.000	437.5	Below standard DH
50	75.0	7.071	530.3	Faster
67	66.5	8.185	544.3	Peak
100	50.0	10.00	500.0	Standard DH

3.3 Normalization: The Ratio Function $R(\alpha)$

The function $f(M)$ still depends on T and N, so it is normalized against the standard-DH baseline, yielding a universal ratio function $R(\alpha)$.

The value of $f(M)$ for standard DH ($M = T$) is:

$$f_{\mathrm{std}} = \left(T - \frac{T}{2}\right)\sqrt{T} = \frac{1}{2}T^{3/2}. \tag{10}$$

For BFDH with fraction α, substituting $M = \alpha T$ gives:

$$f(\alpha T) = \left(T - \frac{\alpha T}{2}\right)\sqrt{\alpha T} = T^{3/2}\left(1 - \frac{\alpha}{2}\right)\sqrt{\alpha}. \tag{11}$$

The efficiency ratio is:

$$R(\alpha) = \frac{f(\alpha T)}{f_{\text{std}}} = \frac{T^{3/2}(1-\alpha/2)\sqrt{\alpha}}{\frac{1}{2}T^{3/2}} = 2\left(1-\frac{\alpha}{2}\right)\sqrt{\alpha}. \tag{12}$$

Expanding this:

$$\boxed{R(\alpha) = 2\sqrt{\alpha} - \alpha\sqrt{\alpha}.} \tag{13}$$

T and N cancel out in equation (12), so $R(\alpha)$ holds for a dataset of any size and any threshold value. Interpretation: $R > 1$ means BFDH is faster than standard DH at this step budget; $R = 1$ means they are equivalent; $R < 1$ means BFDH is slower.

3.4 Critical Points of $R(\alpha)$

Theorem 2 (Critical Points of $R(\alpha)$)**.** *The function $R(\alpha) = 2\sqrt{\alpha} - \alpha\sqrt{\alpha}$ on $\alpha \in (0,1]$ has the following properties:*

1. $\lim_{\alpha\to 0^+} R(\alpha) = 0$.
2. $R(\alpha) = 1$ *at two points:* $\alpha_{\text{inf}} = (3-\sqrt{5})/2 \approx 0.382$ *(lower break-even point) and* $\alpha = 1$ *(upper break-even point, identical to standard DH).*
3. $R(\alpha)$ *attains a global maximum at* $\alpha^* = 2/3$*, with* $R(2/3) = \frac{4}{3}\sqrt{2/3} \approx 1.0887$.

Proof. **Item 1.** Follows directly from $\lim_{\alpha\to 0^+}\sqrt{\alpha} = 0$.

Item 2. Solving $R(\alpha) = 1$:

$$2\sqrt{\alpha} - \alpha\sqrt{\alpha} = 1. \tag{14}$$

Substituting $x = \sqrt{\alpha}$ ($x > 0$) turns this into a cubic polynomial:

$$2x - x^3 = 1 \implies x^3 - 2x + 1 = 0. \tag{15}$$

Trying $x = 1$: $1-2+1 = 0$, so $(x-1)$ is one of its factors. Polynomial division gives:

$$x^3 - 2x + 1 = (x-1)(x^2 + x - 1) = 0. \tag{16}$$

The factor $(x-1)$ gives $x = 1$, i.e., $\alpha = 1$. For the quadratic factor $x^2 + x - 1 = 0$, the quadratic formula gives $x = (-1 \pm \sqrt{5})/2$. Since $x > 0$, we take:

$$x = \frac{-1+\sqrt{5}}{2} \implies \alpha_{\text{inf}} = x^2 = \frac{3-\sqrt{5}}{2} \approx 0.382. \tag{17}$$

As a side note, the value of x in equation (17) happens to equal the inverse golden ratio, $1/\varphi$ with $\varphi = (1+\sqrt{5})/2$; this is simply an algebraic consequence of the cubic in equation (15) and likely carries no deeper physical meaning in the BFDH context.

Item 3. The derivative is $R'(\alpha) = \alpha^{-1/2} - \frac{3}{2}\alpha^{1/2}$. Setting $R'(\alpha) = 0$ gives $\alpha^* = 2/3$, matching $M^* = 2T/3$ as expected, since $\alpha = M/T$. The peak value is:

$$R\left(\frac{2}{3}\right) = \frac{4}{3}\sqrt{\frac{2}{3}} \approx 1.089. \tag{18}$$

□

The curve $R(\alpha)$ is non-monotonic on $\alpha \in (0,1]$: it rises from 0 to a peak at $\alpha^* = 2/3$, then falls back down to $R = 1$ at $\alpha = 1$. The region $\alpha \in (\alpha_{\text{inf}}, 1)$ is where BFDH outperforms standard DH at $S = 1$.

4 Generalizing to Step Budget S

The results in Section 3 hold only for $S = 1$. The natural question is whether the balance point $\alpha^* = 2/3$ remains optimal for a larger *step budget* S, with the same α held constant across every step in that budget.

4.1 Threshold Dynamics Under Step Budget S

Notation follows Section 2, with the addition of a step index k (counted from $k = 0$) within the budget S. At step k, the oracle flags $M_k = \alpha T_k$ elements, with α held constant across the whole budget.

Following equation (1), the threshold at the next step is:

$$T_{k+1} = \frac{M_k}{2} = \frac{\alpha T_k}{2}. \tag{19}$$

Applying equation (19) repeatedly starting from T_0:

$$T_k = T_0\left(\frac{\alpha}{2}\right)^k. \tag{20}$$

The Grover iteration cost at step k, following equation (2), is:

$$I_k = \frac{\pi}{4}\sqrt{\frac{N}{M_k}} = \frac{\pi}{4}\sqrt{\frac{N}{\alpha T_k}}. \tag{21}$$

Substituting equation (20) into equation (21) gives the cost as an explicit function of k:

$$I_k = \frac{\pi}{4}\sqrt{\frac{N}{T_0}}\cdot\alpha^{-1/2}\left(\sqrt{\frac{2}{\alpha}}\right)^k. \tag{22}$$

Define a circuit constant, independent of both α and k:

$$K = \frac{\pi}{4}\sqrt{\frac{N}{T_0}}. \tag{23}$$

With this notation, equation (22) becomes:

$$I_k = K\cdot\alpha^{-1/2}\left(\sqrt{\frac{2}{\alpha}}\right)^k. \tag{24}$$

I_k forms a geometric series in k with first term $K\alpha^{-1/2}$ and ratio $r = \sqrt{2/\alpha}$. This series structure is the foundation for the derivation in Section 5.

5 The Formula $R_S(\alpha)$

5.1 Total Gain Under Step Budget S

The total gain after S steps is $\Delta T_S = T_0 - T_S$. Using equation (20) with $k = S$:

$$\Delta T_S(\alpha) = T_0\left[1 - \left(\frac{\alpha}{2}\right)^S\right]. \tag{25}$$

The term $(\alpha/2)^S$ represents the remaining fraction of the threshold that has not yet been brought down after S steps. The smaller α is, the smaller $(\alpha/2)^S$ becomes, so ΔT_S gets

closer to T_0 (a deeper drop in the threshold). Conversely, as α grows larger (approaching 2), $(\alpha/2)^S$ approaches 1, so ΔT_S approaches 0. This behavior is consistent with the recursion-convergence boundary in equation (20), discussed further in Section 7.

5.2 Total Cost Under Step Budget S

The total cost is the sum of I_k from $k=0$ to $k=S-1$:

$$C_S(\alpha) = K \cdot \alpha^{-1/2} \sum_{k=0}^{S-1} \left(\sqrt{\frac{2}{\alpha}}\right)^k. \tag{26}$$

For $\alpha \neq 2$, this series is summed using the geometric series formula, $\sum_{k=0}^{S-1} r^k = (r^S-1)/(r-1)$:

$$C_S(\alpha) = K \cdot \alpha^{-1/2} \cdot \frac{\left(\sqrt{2/\alpha}\right)^S - 1}{\sqrt{2/\alpha} - 1}. \tag{27}$$

5.3 Absolute Efficiency and the Acceleration Ratio

Absolute efficiency at *step budget* S is:

$$\eta_S(\alpha) = \frac{\Delta T_S(\alpha)}{C_S(\alpha)}. \tag{28}$$

Normalized against standard DH ($\alpha = 1$):

$$R_S(\alpha) = \frac{\eta_S(\alpha)}{\eta_S(1)}. \tag{29}$$

Substituting equations (25) and (26) into equation (28), then into equation (29), and canceling the constants K and T_0 that appear in either the numerator or denominator, yields:

$$\boxed{R_S(\alpha) = \frac{1-(\alpha/2)^S}{1-(1/2)^S} \cdot \frac{\sum_{k=0}^{S-1} (\sqrt{2})^k}{\alpha^{-1/2} \sum_{k=0}^{S-1} \left(\sqrt{2/\alpha}\right)^k}} \tag{30}$$

The first factor is the gain ratio of BFDH relative to standard DH over the same S steps. The second factor is the cost ratio of standard DH to BFDH (numerator and denominator are swapped here since this is the inverse of the cost ratio). $R_S(\alpha) > 1$ means BFDH is more efficient than standard DH within that S-step budget.

Consistency with the $S = 1$ Result

Substituting $S = 1$ into equation (30): both sums reduce to a single term ($k = 0$, equal to 1):

$$R_1(\alpha) = \frac{1-\alpha/2}{1-1/2} \cdot \frac{1}{\alpha^{-1/2}} = 2\sqrt{\alpha} - \alpha\sqrt{\alpha}. \tag{31}$$

This is identical to equation (13), confirming that equation (30) is a genuine generalization of the result in Section 3.

6 Finding the Optimum $\alpha^*(S)$

6.1 Method

The optimum $\alpha^*(S)$ for each *step budget* S is found by solving $R'_S(\alpha) = 0$. The substitution $x = \sqrt{\alpha}$ is used to reduce the derivative to a polynomial in x, the same trick used in item 2 of Theorem 2.

For $S = 1$, this substitution yields a degree-three polynomial that can be solved fully algebraically, giving $\alpha^*(1) = 2/3$ as in equation (31).

For $S \geq 2$, the derivative $R'_S(\alpha) = 0$ produces a polynomial of higher degree. As an example, for $S = 2$, equation (30) reduces to:

$$R_2(\alpha) = \frac{4(1+\sqrt{2})}{3} \cdot \frac{\alpha\left(1 - \frac{\alpha^2}{4}\right)}{\sqrt{\alpha}+\sqrt{2}}. \tag{32}$$

Substituting $\alpha = x^2$ and differentiating with respect to x produces the following degree-five polynomial, once the trivial term $x = 0$ is dropped (irrelevant for root-finding since $x > 0$):

$$5x^5 + 6\sqrt{2}x^4 - 4x - 8\sqrt{2} = 0. \tag{33}$$

For $S \geq 2$ in general, the resulting polynomial reaches degree five or higher as S increases. By the Abel–Ruffini theorem, polynomials of degree five or higher have no general algebraic (radical) solution formula. For $S \geq 2$, $\alpha^*(S)$ is therefore solved numerically, using the Newton–Raphson method on the derivative polynomial, and independently verified through a direct numerical sweep of $R_S(\alpha)$ over the domain $\alpha \in (0,2)$ at high resolution (see Section 8). Both methods agree on $\alpha^*(S)$ to four decimal places.

For $S = 2$, the numerical solution of equation (33) gives $x \approx 1.03142$, so $\alpha^*(2) = x^2 \approx 1.0638$.

6.2 Results

Table 2 shows $\alpha^*(S)$ and the corresponding R_S value at that point, for various *step budgets* S.

Table 2: Optimum $\alpha^*(S)$ for various *step budgets* S.

S	$\alpha^*(S)$	$R_S(\alpha^*)$
1	0.6667	1.0887
2	1.0638	1.0040
3	1.2823	1.1046
4	1.4188	1.3038
5	1.5119	1.5935
10	1.7293	5.326
20	1.8569	89.41
50	1.9408	1.17×10^6
100	1.9700	2.01×10^{13}

At $S = 1$, the optimum sits below 1.0 (the oracle flags a narrower range than the running threshold, matching the name BFDH). Starting at $S = 2$, the optimum crosses past 1.0 (the oracle flags a wider range than the running threshold) — a behavior that remains mathematically consistent with the definition $\alpha = M/T$, which does not restrict α to the interval

$(0,1]$. As S grows larger, the optimum keeps approaching 2.0 from below, and the R_S value at that point grows rapidly. Section 7 proves analytically why 2.0 emerges as the structural bound.

7 Proof of the Limit $\alpha^*(S) \to 2$

The limit $\alpha^*(S) \to 2$ as $S \to \infty$ follows directly from the recursive structure in equation (20):

$$T_S = T_0 \left(\frac{\alpha}{2}\right)^S. \tag{34}$$

There are three possible behaviors of T_S as $S \to \infty$, depending on the value of α:

- If $\alpha < 2$, then $\alpha/2 < 1$, so $T_S \to 0$: the threshold converges to zero.
- If $\alpha = 2$, then $\alpha/2 = 1$, so $T_S = T_0$ for all S: the threshold never drops.
- If $\alpha > 2$, then $\alpha/2 > 1$, so $T_S \to \infty$: the threshold grows without bound, which has no physical use for a minimum-search problem since the threshold would actually be drifting upward, away from the minimum, rather than toward it.

Thus $\alpha = 2$ is the critical convergence boundary of the recursion itself, independent of any consideration of Grover cost.

In the cost given by equation (27), the series ratio $r = \sqrt{2/\alpha}$ is always > 1 for $\alpha < 2$, so the per-step cost grows geometrically as k increases. The closer α gets to 2, the closer r gets to 1 from above, and the slower that per-step cost growth becomes.

The combined effect: pushing α toward 2 makes (a) the per-step gain very small, since T_k barely shrinks, while (b) the per-step cost becomes nearly constant and quite cheap, since $M_k = \alpha T_k$ stays large at every step. For a large *step budget* S, the accumulation of these small, consistent gains, divided by a cost that stays nearly constant and cheap, yields an η_S ratio far larger than a strategy that cuts the threshold aggressively but pays a steep price in its later steps. This is the structural reason $\alpha^*(S)$ keeps getting pushed up toward 2 as S grows, and why it never actually reaches 2, let alone exceeds it.

8 Numerical Experiments

This section presents three numerical experiments that directly verify the central claims of Sections 5 and 7. All experiments were run using Python code (`numpy`, `matplotlib`), and the raw data is included so the results can be independently re-verified.

8.1 Experiment 1: Verifying the Optimum via Direct Sweep

This experiment confirms that the $\alpha^*(S)$ values in Table 2 are genuine global maxima of $R_S(\alpha)$, rather than merely roots of the derivative that could turn out to be saddle points or minima. $R_S(\alpha)$ is computed directly from equation (30) at 20,000 points of α spread evenly over the domain $(0,2)$, for $S \in \{1,2,3,5,10\}$.

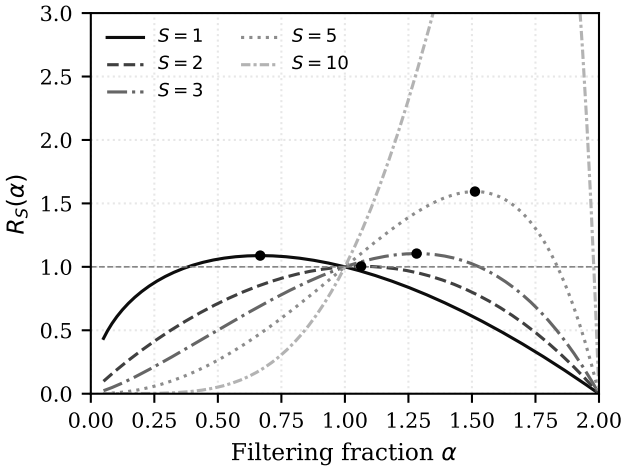

Figure 1: Curves of $R_S(\alpha)$ for several values of S, computed directly from equation (30) at 20,000 points of $\alpha \in (0,2)$. Black dots mark $\alpha^*(S)$ found by numerical search, landing exactly on each curve's peak. The horizontal dashed line marks $R_S = 1$, the performance of standard DH.

Figure 1 shows that every $R_S(\alpha)$ curve has exactly one peak over the tested domain, and that peak's position matches $\alpha^*(S)$ in Table 2 to four decimal places. This independently confirms that the Newton–Raphson method did not mistakenly latch onto a root that is not the global maximum, and it also shows the curve's peak shifting progressively rightward as S grows.

8.2 Experiment 2: Convergence of $\alpha^*(S)$ Toward the Bound $\alpha = 2$

This experiment visually verifies the limit $\alpha^*(S) \to 2$ established in Section 7. The optimum is found through a numerical search over S on a logarithmic range from 1 to 500, covering values of S far larger than those in Table 2.

Figure 2 shows that $\alpha^*(S)$ rises monotonically across the entire tested range of S, with the rate of increase slowing as S grows, consistent with an asymptotic approach to a bound. At $S = 500$, $\alpha^*(500) \approx 1.9939$, only 0.0061 away from the bound $\alpha = 2$ — reminiscent of Zeno's paradox.

8.3 Experiment 3: Behavior of $\alpha^*(S)$ Beyond the Fixed Step Budget

This third experiment illustrates a property of $\alpha^*(S)$ worth keeping in mind when applying the results in Table 2: the value of $\alpha^*(S)$ is defined and optimized specifically for its corresponding *step budget* S, and is not meant to be read outside that budget. As an illustration, take $T_0 = 2^{50}$ and $N = 2T_0$, and compare two candidates at $S = 50$: $\alpha = 1.0$ (standard DH) and $\alpha = 1.93 \approx \alpha^*(50)$.

Within the fixed *step budget* $S = 50$ (panel b, first two bars), $\alpha = 1.93$ is far cheaper than $\alpha = 1.0$ (63.9 versus 9.0×10^7

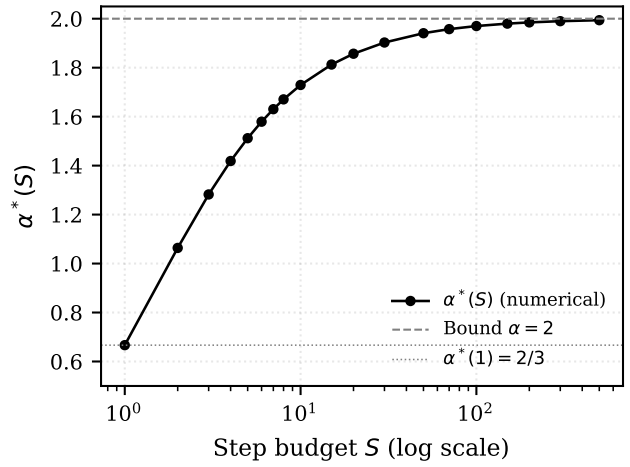

Figure 2: Optimum $\alpha^*(S)$ against *step budget* S, for S from 1 to 500 on a logarithmic scale. The curve rises monotonically and flattens as it approaches the boundary line $\alpha = 2$.

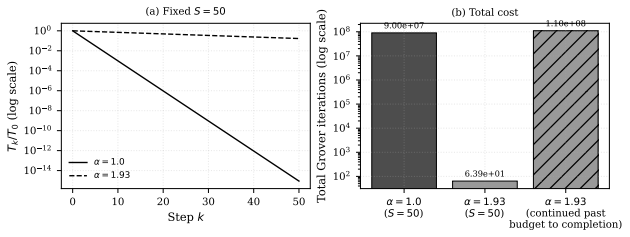

Figure 3: (a) Normalized threshold T_k/T_0 against step k, for $\alpha = 1.0$ and $\alpha = 1.93$ at *step budget* $S = 50$. (b) Total Grover iteration cost (log scale) for three scenarios: $\alpha = 1.0$ at $S = 50$, $\alpha = 1.93$ at $S = 50$, and $\alpha = 1.93$ continued (switching to $\alpha = 1.0$) until T approaches 1. The hatched bar marks the scenario extended beyond the original $S = 50$ budget.

iterations), matching $R_{50} \approx 1.17 \times 10^6$ in Table 2. However, panel (a) shows that by the end of those 50 steps, $\alpha = 1.93$ has only brought the threshold down to about 17% of T_0, while $\alpha = 1.0$ has already reached $T_{50} = 1$. If $\alpha = 1.93$ is continued past its original budget (switching to $\alpha = 1.0$) until $T \approx 1$ is actually reached, an additional 48 steps are needed at an extra cost of roughly 1.10×10^8 iterations (third bar in panel b), bringing the total slightly past the cost of running $\alpha = 1.0$ for $S = 50$ steps.

This does not contradict Table 2; rather, it confirms that $\alpha^*(S)$ answers the question "what is the best α within this budget of S steps," and that answer does not automatically carry over once the budget is extended beyond the originally fixed S — a genuinely different context.

9 How the Threshold-Cutting Pattern Changes with S

It is also worth noting how the qualitative shape of the threshold's descent changes depending on the chosen $\alpha^*(S)$.

At $\alpha = 1.0$ (standard DH, suited to small S), every step cuts the threshold by exactly 50%, similar to halving the search space in binary search. The per-step cost rises sharply as k increases, since M_k shrinks quickly.

At α close to 2.0 (suited to large S), each step cuts the threshold by only a few percent, and that percentage stays roughly constant from step to step. The per-step cost barely changes, since $M_k = \alpha T_k$ stays large throughout the entire budget.

This qualitative difference shows that the correlation between M and DP does not just shift its optimum quantitatively with S; it also changes the *underlying shape of the strategy* itself, moving from an aggressive, binary-search-like cut at small *step budgets* toward a gentle, incremental decline at large *step budgets*.

10 Conclusion

This paper defines and analyzes Block-Filtering Dürr–Hoyer (BFDH), a hypothesis-testing analytical tool for the correlation between M and *drop point* in the Dürr–Hoyer minimum-search algorithm, with all analysis carried out under a fixed *step budget* S.

At $S = 1$, the efficiency function $f(M) = (T - M/2)\sqrt{M}$ is derived from first principles and maximized at $M^* = 2T/3$. Its normalized form, the universal ratio function $R(\alpha) = 2\sqrt{\alpha} - \alpha\sqrt{\alpha}$, reaches a global peak at $\alpha^* = 2/3$ with an 8.87% speedup over standard DH, and a lower break-even point at $\alpha_{\text{inf}} = (3 - \sqrt{5})/2 \approx 0.382$.

Generalizing to *step budget* S, the formula $R_S(\alpha)$ in equation (30) consistently reduces to the $S = 1$ result. The central finding of this generalization is that the optimum $\alpha^*(S)$ is not a single constant, but a monotonically increasing function of S, moving from $2/3$ at $S = 1$ toward the structural bound 2 as $S \to \infty$, with that bound emerging directly from the convergence condition of the threshold recursion. For $S \geq 2$, this optimum has no closed algebraic form (a consequence of the Abel–Ruffini theorem applied to derivative polynomials of degree five or higher) and is solved numerically, with results verified through the experiments in Section 8.

These results establish that the correlation between M and *drop point* is curved and tied to whichever *step budget* S is under consideration: there is no single balance point that holds for every S. As a direct consequence, the question "what is the optimal filtering fraction" can only be answered clearly once paired with "optimal for a budget of how many steps."

References

[1] L. K. Grover, "A fast quantum mechanical algorithm for database search," *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 212–219, 1996. https://arxiv.org/abs/quant-ph/9605043

[2] C. Dürr and P. Høyer, "A quantum algorithm for finding the minimum," arXiv:quant-ph/9607014, 1996. https://arxiv.org/abs/quant-ph/9607014

[3] M. Boyer, G. Brassard, P. Høyer, and A. Tapp, "Tight bounds on quantum searching," *Fortschritte der Physik*, vol. 46, no. 4–5, pp. 493–505, 1998. https://arxiv.org/abs/quant-ph/9605034

The LaTeX mathematical notation in this paper was prepared with the help of an AI language model (GPT) as a conversion tool from handwritten notes to LaTeX code, without altering the mathematical substance that I developed. I wrote the notation by hand on paper, then photographed and converted it into LaTeX code with AI assistance. All mathematical content is my own work. IMPORTANT. The original paper is in Indonesian. This version is merely the result of machine translation into English; apologies if the wording is stiff.